BusinessIntelligence.ai

KPI Intelligence-to-Action Engine

Technical Architecture, Implementation Specification & Execution Guide

Team BusinessIntelligence.ai

Accenture Innovation Challenge 2026 — Round 2 Prototype

August 2026

Abstract

Traditional business intelligence dashboards report metric variances (e.g., "Revenue dropped 8% in West Region") but consistently fail to explain *why* the movement occurred or *what concrete actions* decision-makers should take. The resulting translation gap demands days of manual analyst investigation. This document specifies the architecture, algorithmic design, and execution of **BusinessIntelligence.ai**, a production-ready KPI Intelligence-to-Action Engine. The system couples deterministic, SQL-based statistical anomaly detection and additive driver decomposition with an LLM narrative synthesis layer (Gemini 2.0 Flash). Under our strict "No-Hallucination" architectural paradigm, large language models are prohibited from performing quantitative calculations or directly accessing raw database rows, guaranteeing 100% mathematical auditability while delivering persona-calibrated narratives, honest uncertainty abstentions, and role-governed action recommendations.

Contents

1 Implementation Approach

1.1 Core Design Principle: The No-Hallucination Architecture

The central design requirement established by the problem statement is that **the Large Language Model (LLM) must never be treated as the source of quantitative truth.**

In conventional generative analytics architectures, LLMs are frequently prompted to aggregate numbers or interpret raw transactional tables, introducing substantial risks of arithmetic errors, hallucinated percentages, and unverified causal claims. Our architecture strictly separates:

1. **Deterministic Analytical Plane (Non-LLM):** Handles metric computation via SQL, Z-score anomaly detection, mathematical additive decomposition, rule-based confidence calibration, and role-based data entitlement filtering.

2. **Generative Narrative Plane (LLM):** Receives only pre-aggregated, statistically validated JSON summaries to generate persona-adapted prose, format structured recommendations, and articulate abstention reasoning.

The 10-Step Pipeline Execution Flow

1. **Contract Resolution:** Runtime loads declarative YAML semantic contracts specifying SQL expressions, lineages, and alert thresholds.
2. **Parameterized SQL Execution:** SQLite executes deterministic queries parameterized with temporal filters and RBAC regional constraints.
3. **Statistical Anomaly Evaluation:** Compares movements against contractual percentage and business impact thresholds ($Z > 2.5$).
4. **Additive Driver Decomposition:** Decomposes metric variance into Volume, Price, and Mix components ($\Delta\text{Rev} = \Delta V + \Delta P + \Delta M$).
5. **Heuristic Confidence Scoring:** Evaluates sample maturity, history duration, and cross-source consistency ($0.0 \leq S \leq 1.0$).
6. **Uncertainty & Abstention Gate:** If confidence $S < 0.40$, halts processing and returns an explicit, calibrated abstention notice.
7. **LLM Context Serialization:** Packs deterministic analytical output into structured prompt context.
8. **Persona-Calibrated Synthesis:** Invokes Gemini 2.0 Flash with specialized persona system prompts (CEO, Analyst, Regional Manager).
9. **Structured Action Generation:** LLM returns structured action arrays: Driver → Lever → Action → Impact → Owner.
10. **Telemetry & Evidence Logging:** Records execution latency, token consumption, estimated USD cost, and source lineage.

1.2 Addressing Real-World Operational Complexities

Complexity	Architectural Resolution
Interacting Drivers	Additive variance decomposition separating unit volume from price realization and product/channel mix.
Heterogeneous Cadences	Unified relational storage reconciling daily transactional grain, weekly marketing logs, and monthly P&L.
Sparse History / Cold Start	Declarative `sparse_history` flags in semantic contracts; automatically suppresses anomaly models until baseline window is satisfied.
Contradictory / Ambiguous Data	Confidence scoring engine triggers programmatic abstentions with explicit diagnostic justifications.
Multi-Tenant Security / RBAC	Dual-layer entitlement: SQL-level `WHERE` clause parameterization and persona prompt containment.
Operational Cost & Latency	Per-request runtime telemetry tracking token volume, microsecond latency breakdown, and inference costs.

Table 1: Operational Complexity Handling Matrix

2 Solution Architecture

2.1 Layered System Topology

The system is constructed as a decoupled, multi-tier service architecture:

```
+--------------------------------------------------------------------------+
|                     Presentation Layer (Vite / Vanilla JS)               |
|   * Persona Selector    * KPI Matrix & Sparklines    * Variance Waterfall|
|   * AI Synthesis Panel  * Structured Action Board    * Live Telemetry    |
+------------------------------------+-------------------------------------+
                                     | JSON over HTTP (Proxy :5173 -> :3001)
+------------------------------------v-------------------------------------+
|                    API Gateway & Controller Layer (Express)              |
|   /api/kpis * /api/analysis/:id * /api/narrative/:id * /api/telemetry    |
+-------------------+---------------------------------+--------------------+
                    |                                 |
+-------------------v--------------+   +--------------v--------------------+
|   Deterministic Analytical Core  |   |   Generative Narrative Engine     |
|  * kpiEngine.js (SQL Core)       |   |  * geminiClient.js (Gemini SDK)   |
|  * anomalyDetector.js (Z-Score)  |   |  * narrativeGenerator.js          |
|  * contributionAnalysis.js (Math)|   |  * actionRecommender.js (JSON)    |
|  * confidenceScorer.js (Heuristic|   |  * abstentionHandler.js           |
+-------------------+--------------+   +--------------^--------------------+
                    | Deterministic Analysis Summary  |
                    +---------------------------------+
                                     | Parameterized Queries & Audit Log
+------------------------------------v-------------------------------------+
|                    Relational Storage Layer (SQLite WAL)                 |
|   * sales_daily (4,992 rows)       * marketing_weekly (159 rows)         |
|   * financials_monthly (12 rows)   * feedback & telemetry tables         |
+--------------------------------------------------------------------------+
```

2.2 Data Source Architecture and Grain Reconciliation

The engine synthesizes three distinct internal data feeds seeded across 12 months (2025):

1. **Source A: Daily Sales (`sales_daily`):** Grain: `[Date, Product, Region, Channel]`. Captures transactional volume, realized unit price, revenue, COGS, gross profit, website traffic sessions, and customer acquisition.
2. **Source B: Weekly Marketing Spend (`marketing_weekly`):** Grain: `[Week, Channel]`. Captures channel spend across Online, Offline, and Partner channels, impression volumes, clicks, and CTR.
3. **Source C: Monthly Financials (`financials_monthly`):** Grain: `[Month]`. Captures aggregate corporate OPEX, EBITDA, headcount, qualitative external events log, and data quality flags.

3 Deterministic Analytical Core (Non-LLM)

3.1 Mathematical Additive Variance Decomposition

For revenue analysis, the engine decomposes the total variance between current period (t) and prior period ($t-1$) into exact constituent components:

$$\Delta\text{Revenue} = \text{Revenue}_t - \text{Revenue}_{t-1} \tag{1}$$
$$\text{Volume Effect} = (Q_t - Q_{t-1}) \times \bar{P}_{t-1} \tag{2}$$
$$\text{Price Effect} = Q_t \times (\bar{P}_t - \bar{P}_{t-1}) \tag{3}$$
$$\text{Mix/Channel Effect} = \Delta\text{Revenue} - (\text{Volume Effect} + \text{Price Effect}) \tag{4}$$

where Q denotes units sold and $\bar{P}$ denotes average realized selling price. This formulation guarantees that the driver contributions sum to exactly 100% of the observed revenue delta without mathematical residuals.

3.2 Statistical Materiality & Anomaly Detection

A metric movement is flagged as anomalous if and only if:

$$(|\%\Delta| \geq \tau_{\text{pct}} \quad \vee \quad |\Delta\text{Value}| \geq \tau_{\text{USD}}) \quad \wedge \quad |Z| \geq 2.5 \tag{5}$$

where thresholds τ_{pct} and τ_{USD} are governed declaratively by the KPI contract.

3.3 Confidence Calibration & Programmatic Abstention

The confidence scoring engine calculates an evidence reliability index $S \in [0, 1]$:

- **High Confidence ($S = 0.95$):** Complete historical data, multi-source corroboration, no qualitative data flags.
- **Low Confidence ($S = 0.35$):** Incomplete attribution windows (e.g., CAC measured within < 14 days of a major campaign surge).
- **Very Low Confidence ($S = 0.10$):** Sparse history (< 8 baseline weeks for newly launched products).

When $S < 0.40$, the engine activates the `abstentionHandler`, bypassing generative synthesis and returning a structured abstention notice with explicit remediation guidance.

4 Generative Narrative & Action Layer (LLM)

4.1 Persona-Specific System Prompting

When analytical confidence permits synthesis, pre-computed driver summaries are passed to Gemini 2.0 Flash with strictly constrained role instructions:

- **CEO Persona:** Focuses on macro bottom-line variance, strategic trade-offs, and single highest-leverage executive intervention. Max 3–4 sentences.

- **Analyst Persona:** Emphasizes mathematical decomposition percentages, Z-score significance, data lineage, and supplier feed data quality caveats.

- **Regional Manager Persona:** Strict spatial containment (e.g., North Region only). Emphasizes regional sales volumes, local channel mix, and regional branch levers.

4.2 Structured Action Recommendation Schema

Actions are generated in raw JSON according to a standardized schema:

```
[
  {
    "driver": "Volume Decline (West)",
    "lever": "Supply Chain / Inventory Allocation",
    "action": "Expedite inter-warehouse inventory transfer from Central
        to West hubs",
    "expected_impact": "+$180K revenue recovery over 14 days",
    "owner": "VP Supply Chain"
  }
]
```

5 KPI Semantic Contracts

KPI metadata is maintained in declarative YAML definitions within `kpi-contracts/`:

KPI ID	Formula Expression	Grain	Threshold	Allowed Roles
`revenue`	$\sum(\text{units} \times \text{price})$	Daily	±5% / $50K	CEO, Analyst, Reg Mgr
`gross_margin`	(Gross Profit/Rev) × 100	Monthly	±2 pp / $30K	CEO, Analyst
`cac`	Spend/New Customers	Monthly	±15% / $10K	CEO, Analyst
`conversion_rate`	(Units/Sessions) × 100	Daily	±1 pp	All Personas
`new_product_gmv`	$\sum \text{Rev}_{\text{new\_product}=1}$	Daily	±20%	CEO, Analyst

Table 2: Monitored KPI Specifications

6 Role-Based Access Control (RBAC)

Security and data entitlement are enforced across two decoupled boundaries:

1. **SQL Query Layer:** Injects parameter-safe filters directly into the statement AST (e.g., `WHERE region = 'North'`).
2. **Prompt Context Layer:** Injects role constraints into the LLM system prompt, explicitly forbidding references to unauthorized regions or corporate-only metrics.

Role Identifier	Permitted KPIs	Regional Filter	Granularity
`ceo`	All 5 KPIs	Global (All)	Monthly Aggregates
`analyst`	All 5 KPIs	Global (All)	Full Daily Granularity
`regional_manager_north`	Revenue, Conversion Rate	North Region Only	Regional Daily
`regional_manager_south`	Revenue, Conversion Rate	South Region Only	Regional Daily

Table 3: Role Entitlement Matrix

7 System Dependencies & Execution Guide

7.1 Environment & Software Prerequisites

- **Node.js:** Version ≥ 18.0.0 (LTS recommended)
- **Python 3:** Version ≥ 3.8 (used exclusively for synthetic data generation)
- **npm:** Version ≥ 8.0.0

7.2 Backend Dependencies

- `express` (ˆ4.21.0): REST routing framework
- `better-sqlite3` (ˆ11.0.0): High-performance synchronous SQLite driver
- `@google/genai` (ˆ1.0.0): Official Google Gen AI SDK for Gemini models
- `js-yaml` (ˆ4.1.0): Declarative contract parser
- `csv-parse` (ˆ5.5.6): High-throughput streaming CSV parser
- `cors`, `dotenv`, `uuid`: Infrastructure utilities

7.3 Frontend Dependencies

- vite (^5.4.1): Development server with HMR and production bundler.
- Zero external UI frameworks (Vanilla ECMAScript 2022 + CSS Tokens).

7.4 Step-by-Step Execution Instructions

7.4.1 Step 1: Generate Deterministic Seed Data

```
# From repository root
python3 scripts/generate_seed_data.py
```

Generates `data/seed/sales_daily.csv` (4,992 rows), `marketing_weekly.csv` (159 rows), and `financials_monthly.csv` (12 rows).

7.4.2 Step 2: Initialize Launch Backend API

```
cd backend
npm install
cp .env.example .env
# Edit .env and supply GEMINI_API_KEY (optional: fallback mock active
   if omitted)
npm run seed      # Builds SQLite database from CSV fixtures
npm run dev       # Launches API service on http://localhost:3001
```

7.4.3 Step 3: Launch Presentation Dashboard

```
# Open a secondary terminal
cd frontend
npm install
npm run dev       # Launches Vite interface on http://localhost:5173
```

Navigate to http://localhost:5173 to inspect the active intelligence interface.

8 API Reference Specification

All endpoints accept the standard header `X-Persona: <role>` and query parameters `year=2025&month=10`.

Method	Endpoint	Description
GET	/health	Service health, version, and server timestamp.
GET	/api/personas	Enumerates available roles and entitlement metadata.
GET	/api/sources	Ingestion freshness timestamps and row counts.
GET	/api/kpis	Evaluates all accessible KPIs for the caller persona.
GET	/api/kpis/:id	Returns single KPI valuation, formula, and lineage.
GET	/api/analysis/:id	Returns statistical anomaly state and driver decomposition.
GET	/api/narrative/:id	Synthesizes persona narrative, actions, and telemetry.
POST	/api/feedback	Records analyst rating (thumbs_up/down) and corrections.
GET	/api/feedback	Returns audit log of feedback entries (Analyst role required).
GET	/api/telemetry	Returns recent request latency, token counts, and cost logs.
GET	/api/telemetry/stats	Aggregate cost, token consumption, and latency metrics.

Table 4: REST API Endpoint Reference

9 Verified Hackathon Demo Scenarios

1. **Scenario 1: Multi-Factor Revenue Drop (Month 10):**
   - *Observed Change:* Revenue drops −33.56% from \$2.14M to \$1.42M ($Z = -3.36$).
   - *Decomposition:* Volume Effect (84.4% / −\$717K), Price Effect (15.6% / −\$150K), Mix Effect (−\$32K).
   - *Outcome:* CEO receives macro executive guidance; Analyst receives exact statistical variance; Regional Manager North receives only North-isolated decline (−\$148K).
2. **Scenario 2: Low-Confidence CAC Abstention:**
   - *Observed Change:* CAC increases +88.04% to \$210.93.
   - *Analytical Gate:* Attribution window incomplete ($S = 0.35$).
   - *Outcome:* Engine abstains with diagnostic message, preventing premature marketing budget cuts.
3. **Scenario 3: Sparse-History Cold Start:**
   - *Observed Change:* New Product GMV (AI Insights Module launched Aug 2025).
   - *Analytical Gate:* History duration < 8 weeks ($S = 0.10$).
   - *Outcome:* Anomaly detection suppressed; interface renders raw trend with a clear `SPARSE` tag.
4. **Scenario 4: Dual-Layer Security Enforcement:**
   - *Action:* Caller assumes `regional_manager_north`.
   - *Outcome:* Gross Margin and CAC cards vanish; direct invocation to `/api/kpis/gross_margin` returns **HTTP 403 Forbidden**.
5. **Scenario 5: Closed-Loop Human Feedback:**
   - *Action:* Analyst submits correction: *"Supplier delay affected West in W3"*.
   - *Outcome:* Persisted in SQLite feedback store with narrative ID linkage for offline prompt fine-tuning.